

# Computational Linguistics 1 (CL3.101) – Course Project Guidelines (2026)

## Project Guidelines (Important – Read First)

- This is a three-month project, to be done in groups of two students.
- Each project should clearly connect linguistic theory with computational implementation.
- The minimum expectation is a *working prototype* with linguistic analysis and error discussion.
- Students are strongly encouraged to work on Indian languages, especially low-resource languages.
- Projects should prioritize linguistic motivation and clarity over heavy machine learning.
- Working beyond the defined scope is welcome, provided the core objectives are clearly met.
- Students may:
  - o Extend their project to additional languages
  - o Compare linguistic frameworks
  - o Add optional ML-based experiments
- Any extension must be clearly marked as “beyond scope” in the final report.
- A project proposal (2–3 pages) must be approved by the instructors.

| Project No | Project Title                                  | Primary Units | Core Task (Minimum Scope)                       | Expected Output     |
|------------|------------------------------------------------|---------------|-------------------------------------------------|---------------------|
| P1         | Tokenizer & Normalizer for an Indian Language  | Unit 1        | Script-aware normalization and tokenization     | Tool + evaluation   |
| P2         | Zipf’s Law Analysis of Indian Languages Corpus | Unit 1        | Corpus statistics and linguistic interpretation | Report + plots      |
| P3         | Regex-based Linguistic Pattern Identifier      | Unit 1        | Identify language-specific patterns using regex | Rule set + examples |

|     |                                        |            |                                                     |                           |
|-----|----------------------------------------|------------|-----------------------------------------------------|---------------------------|
| P4  | Rule-based Morphological Analyzer      | Unit 2     | Analyze inflectional morphology for nouns/verbs     | Analyzer + error analysis |
| P5  | Finite-State Morph Analyzer using FST  | Unit 2     | Implement morphology using FSA/FST                  | FST + coverage report     |
| P6  | Morphological Generator from Paradigms | Unit 2     | Generate full paradigm for selected word classes    | Generator + paradigms     |
| P7  | POS Tagset Design & Annotation         | Unit 3     | Create tagset and annotate ~1K tokens               | Tagset + annotated data   |
| P8  | Rule-based POS Tagger                  | Unit 3     | Develop rule-based tagger with suffix/context rules | Tagger + evaluation       |
| P9  | Shallow Parser / Chunker               | Unit 3     | NP/VP or LWG chunking using rules                   | Chunker + examples        |
| P10 | Multiword Expression Identification    | Unit 3     | Identify idioms/compound verbs/NEs                  | MWE list + detector       |
| P11 | CFG-based Syntax Parser                | Unit 4     | Write grammar rules and parse simple sentences      | Grammar + parser          |
| P12 | Dependency Annotation Study            | Unit 4     | Annotate sentences using UD or Paninian grammar     | Annotated corpus          |
| P13 | Paninian Grammar Analysis Tool         | Unit 4     | Karaka & vibhakti mapping for sentences             | Analyzer + charts         |
| P14 | Paninian vs UD Comparative Study       | Unit 4     | Compare two frameworks on same data                 | Comparative report        |
| P15 | Speech Corpus Creation & Analysis      | Unit 5     | Create small speech corpus + phonetic analysis      | Corpus + documentation    |
| P16 | Grapheme-to-Phoneme Mapping            | Unit 5     | Rule-based G2P for Indian language                  | G2P rules + evaluation    |
| P17 | End-to-End Linguistic Pipeline         | Units 1–4  | Tokenization → Morphology → POS → Chunking          | Modular pipeline          |
| P18 | Code-Mixed Text Analysis               | Units 2–3  | Analyze morphology/POS issues in code-mix           | Analysis + prototype      |
| P19 | Linguistic Resource Creation           | Cross-unit | Build lexicon/paradigms/annotated sentences         | Reusable resource         |